

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN THIẾT KẾ HỆ THỐNG SOC

**THIẾT KẾ VÀ TRIỂN KHAI BỘ BẮM SHA-256 BẰNG NGÔN
NGỮ MÔ TẢ PHẦN CỨNG VERILOG TRÊN KIT DE2**

GVHD: ThS. Trương Văn Cường

Lớp: CE433.Q21

Sinh viên thực hiện:

Dương Trọng Nghĩa – 23521007

Hà Trọng Nghĩa – 23521008

Nguyễn Trường Lâm – 23520835

TP. HỒ CHÍ MINH – 2026

Phụ lục

.....	1
DANH MỤC HÌNH ẢNH	3
DANH MỤC BẢNG BIỂU	3
THÔNG TIN NHÓM THỰC HIỆN.....	4
QUÁ TRÌNH LÀM VIỆC NHÓM	5
LỜI CẢM ƠN.....	6
Chương 1: MỞ ĐẦU	7
1.1. Đặt vấn đề và Lý do chọn đề tài	7
1.2. Mục tiêu nghiên cứu.....	7
1.3. Đối tượng, Phạm vi và Phương pháp	7
Chương 2: TỔNG QUAN HỆ THỐNG VÀ THIẾT KẾ PHẦN CỨNG SHA-256.....	8
2.1. Thuật toán SHA256	8
2.1.1. Đặc tả thuật toán	8
2.1.2. Quy trình đệm dữ liệu (sha_message_padding)	8
2.1.3. Quy trình xử lý trong khối Core (sha256_core):.....	11
2.1.4. Khối tra cứu hằng số vòng (sha256_constants_k):	13
2.1.5. Bộ nhớ lịch trình thông điệp (sha256_mem):	14
2.1.6. Mạch tổ hợp vòng băm (sha256_round_comb):	15
2.1.7. Thiết kế module top (sha256_top):	17
2.1.8. Thiết kế module Avalon (sha_avalon_top):	17
2.2. Thiết kế hệ thống Qsys	18
2.3. Thiết kế phần mềm.....	20
Chương 3: MÔ PHỎNG VÀ TỔNG HỢP	21
3.1. Kết quả mô phỏng	21
3.1.1. Thiết lập kịch bản kiểm chứng	21
3.1.2. Phân tích dạng sóng và kết quả đầu ra	21
3.2. Kết quả tổng hợp.....	22
3.2.1. Phân tích tài nguyên hệ thống	22
3.2.2. Đánh giá thời gian thực.....	22
3.2.3. Hiệu năng tính toán.....	23
3.2.4. Thông lượng	24
3.3. Mô phỏng và kiểm chứng thực thi trên vi xử lý Nios II.....	24
3.4. Mô phỏng trên kit :.....	25

Chương 4: KẾT QUẢ VÀ ĐÁNH GIÁ	28
4.1. Kết quả đạt được	28
4.2. Hạn chế - Khó khăn.....	28
Chương 5: HƯỚNG PHÁT TRIỂN	29
Chương 6: KẾT LUẬN	30
TÀI LIỆU THAM KHẢO	31

DANH MỤC HÌNH ẢNH

Hình 1. Tổng quan hệ thống	8
Hình 2. Sơ đồ FSM điều khiển bộ đệm dữ liệu	9
Hình 3. Sơ đồ khối sha_message_controller.....	11
Hình 4. Sơ đồ FSM điều khiển lỗi băm SHA-256.....	11
Hình 5. Sơ đồ khối sha256_core.....	13
Hình 6. Sơ đồ khối sha256_constants_k.....	14
Hình 7. Sơ đồ khối sha256_mem.....	15
Hình 8. Sơ đồ khối sha256_round_comb	16
Hình 9. Sơ đồ khối sha256_top.....	17
Hình 10. Sơ đồ khối sha256_avalon_top.....	18
Hình 11. Sơ đồ hệ thống Qsys	19
Hình 12. Sóng waveform	21
Hình 13. Worst Negative Slack	23
Hình 14. Chỉ số Fmax	23
Hình 15. Worst Hold Slack.....	23
Hình 16. Demo 3 bytes ký tự abc (Single-Block).....	25
Hình 17. Demo 3 bytes ký tự abc hiện trên console (Single-Block)	25
Hình 18. Demo 55 bytes toàn ký tự a (Boundary Single-Block).....	26
Hình 19. Demo 55 bytes toàn ký tự a hiện trên console (Boundary Single-Block) .	26
Hình 20. Demo 65 bytes toàn ký tự a (Multi-Block).....	27
Hình 21. Demo 65 bytes toàn ký tự a hiện trên console (Multi-Block)	27

DANH MỤC BẢNG BIỂU

Bảng 1: Thông tin nhóm thực hiện	4
Bảng 2: Bảng quá trình làm việc nhóm	5
Bảng 3: Bảng khai báo tài nguyên sử dụng	22

THÔNG TIN NHÓM THỰC HIỆN

- **Lớp:** CE433.Q21
- **Giảng viên hướng dẫn:** Trương Văn Cương

SỐ THỨ TỰ	HỌ VÀ TÊN	MÃ SỐ SINH VIÊN	NHIỆM VỤ
1	Dương Trọng Nghĩa	23521007	Thành viên
2	Hà Trọng Nghĩa	23521008	Thành viên
3	Nguyễn Trường Lâm	23520835	Thành viên

Bảng 1: Thông tin nhóm thực hiện

QUÁ TRÌNH LÀM VIỆC NHÓM

- Chủ đề nghiên cứu: Thiết kế và triển khai bộ băm sha-256 bằng ngôn ngữ mô tả phần cứng verilog trên kit DE2

- Thời gian thực hiện: từ ngày 21/03/2026 đến ngày 22/05/2026

Thời gian hợp	Nội dung	Kết quả đạt được	Hạn chế
14/03/2026	Phân công công việc, tìm hiểu thuật toán SHA256	Phân chia đồng đều công việc. Có kế hoạch chuẩn bị.	Không có
21/03/2026	Báo cáo tiến độ tìm hiểu công việc. Phân công làm các phần module của thuật toán	Hoàn thành tìm hiểu công việc.	Không có
06/04/2026	Báo cáo kết quả và kiểm thử functional của các module đã làm.	Các module chạy đúng function, hash đúng kết quả.	Timing còn kém do chưa pipeline.
14/04/2026	Phân công tìm hiểu và đọc các bài báo nghiên cứu khoa học để tham khảo cách pipeline cho thuật toán.	Tìm được các ý tưởng giúp cho thuật toán hiệu quả hơn.	Không có
21/04/2026	Áp dụng những cách pipeline tìm hiểu được vào thuật toán SHA256.	Đã có pipeline nên timing được cải thiện tốt hơn.	Timing vẫn chưa đạt mong muốn.
06/05/2026	Tiến hành kiểm thử và tối ưu timing lần cuối.	Đã đạt timing tốt nhất trong khả năng.	Không có
15/05/2026	Viết báo cáo hoàn thành công việc.	Phân chia công việc viết báo cáo và làm slide.	Không có
20/05/2026	Hoàn thành đồ án		

Bảng 2: Bảng quá trình làm việc nhóm

LỜI CẢM ƠN

Nhóm nghiên cứu xin chân thành cảm ơn Quý Thầy Cô, các Khoa và Phòng ban của Trường Đại học Công nghệ Thông tin – ĐHQG TP.HCM đã tạo điều kiện thuận lợi để nhóm hoàn thành Đồ án môn Thiết kế hệ SoC.

Đặc biệt, nhóm nghiên cứu xin gửi lời cảm ơn sâu sắc đến Thầy **Trương Văn Cương** – người đã trực tiếp giảng dạy, hướng dẫn và đồng hành cùng nhóm với sự tận tâm và trách nhiệm. Những kiến thức và sự hỗ trợ từ thầy là nguồn động lực lớn giúp nhóm hoàn thành tốt đồ án.

Nhóm nghiên cứu cũng trân trọng cảm ơn các anh chị và bạn sinh viên trong trường đã nhiệt tình góp ý, chia sẻ kinh nghiệm và hỗ trợ nhóm trong suốt quá trình thực hiện.

Một lần nữa, nhóm xin chân thành cảm ơn!

Tp.Hồ Chí Minh, ngày 21 tháng 5 năm 2026

Đại diện nhóm sinh viên thực hiện.

Dương Trọng Nghĩa

Chương 1: MỞ ĐẦU

1.1. Đặt vấn đề và Lý do chọn đề tài

Trong kỷ nguyên số, thuật toán băm SHA-256 (chuẩn FIPS 180-4) đóng vai trò nền tảng cho an toàn thông tin. Tuy nhiên, việc thực thi bằng phần mềm thường bộc lộ điểm nghẽn về tốc độ và độ trễ khi xử lý dữ liệu lớn. Nhằm khắc phục hạn chế của kiến trúc tuần tự, đề tài “Hiện thực bộ băm SHA-256 bằng ngôn ngữ mô tả phần cứng Verilog” được thực hiện để xây dựng bộ tăng tốc phần cứng chuyên dụng trên FPGA, khai thác tối đa thông lượng tính toán song song.

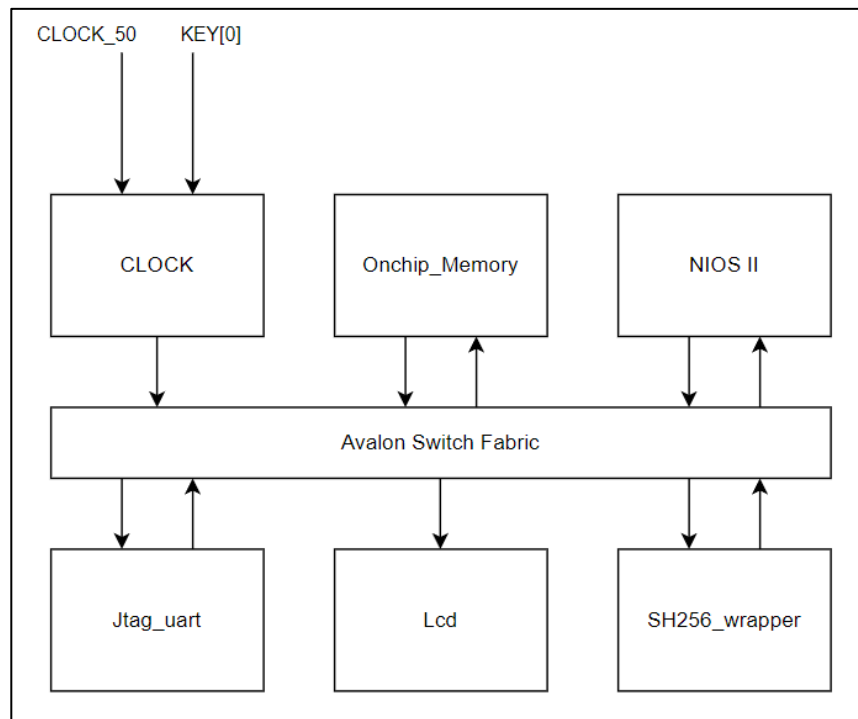
1.2. Mục tiêu nghiên cứu

Mục tiêu cốt lõi của đề án là thiết kế, kiểm chứng và tổng hợp lõi IP-Core SHA-256. Hệ thống tối ưu hóa kiến trúc bằng cách xử lý song song 2 vòng (round) tính toán trong một chu kỳ xung nhịp nhằm gia tăng thông lượng. Thiết kế đảm bảo độ chuẩn xác tuyệt đối theo chuẩn FIPS 180-4 và sẵn sàng tích hợp linh hoạt vào các hệ thống SoC yêu cầu bảo mật tốc độ cao.

1.3. Đối tượng, Phạm vi và Phương pháp

Đối tượng nghiên cứu là thuật toán SHA-256 và thiết kế vi mạch số mức RTL bằng Verilog HDL. Phạm vi đề tài tập trung mô phỏng, hiện thực và kiểm chứng lõi gia tốc độc lập. Tính đúng đắn được xác minh qua RTL Simulation với các test vector chuẩn. Cuối cùng, hệ thống được tổng hợp trên nền tảng Quartus (kit DE2) để đánh giá tiêu thụ tài nguyên (LUTs, FFs), tần số tối đa (Fmax) và thông lượng thực tế.

Chương 2: TỔNG QUAN HỆ THỐNG VÀ THIẾT KẾ PHẦN CỨNG SHA-256



Hình 1. Tổng quan hệ thống

2.1. Thuật toán SHA256

2.1.1. Đặc tả thuật toán

SHA-256 là thuật toán băm mật mã thuộc họ SHA-2, được chuẩn hóa theo tài liệu FIPS 180-4 của Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST). Thuật toán hoạt động dựa trên các thông số kỹ thuật cốt lõi sau:

- Kích thước khối dữ liệu xử lý (Block size): 512 bit.
- Kích thước mã băm đầu ra (Digest size): 256 bit.
- Kích thước từ toán học (Word size): 32 bit.
- Số vòng lặp nén (Rounds): 64 vòng.
- Chiều dài thông điệp tối đa: $2^{64} - 1$ bit.

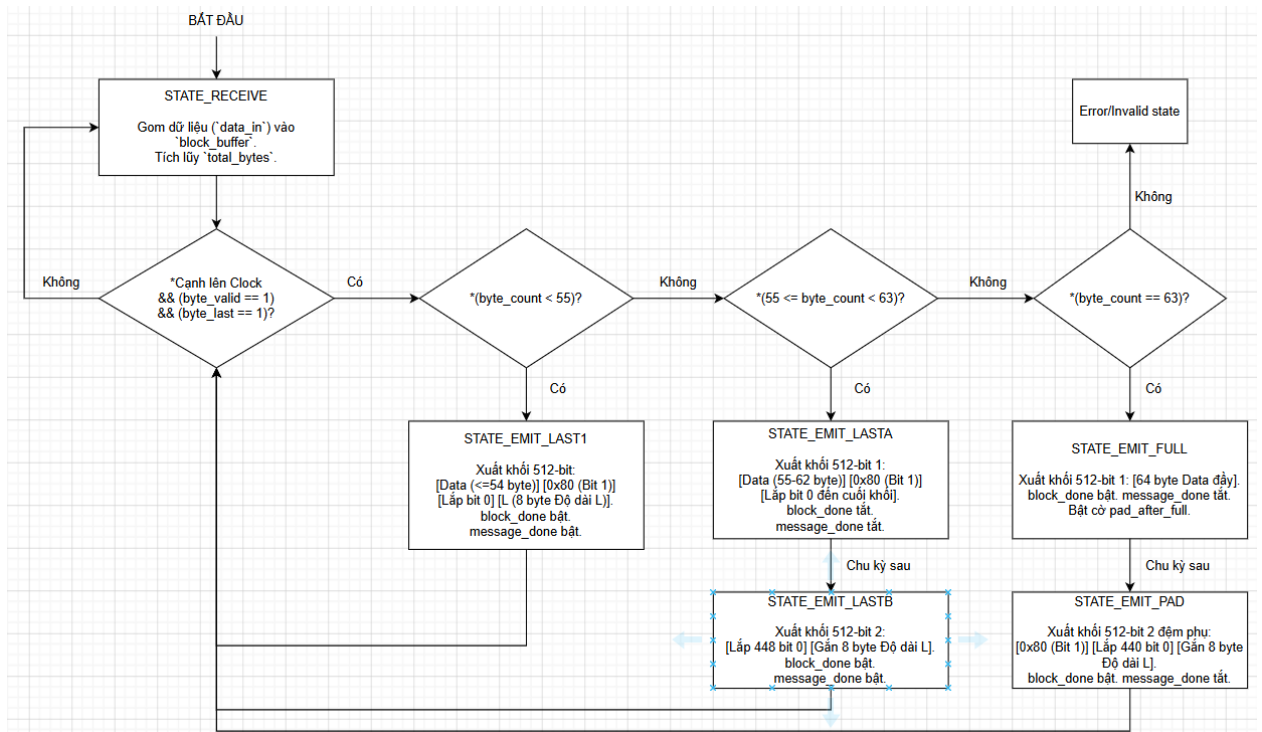
2.1.2. Quy trình đệm dữ liệu (sha_message_padding)

Để luồng dữ liệu đầu vào tương thích với kiến trúc xử lý khối 512-bit, mọi thông điệp gốc có độ dài L bit đều phải trải qua quá trình tiền xử lý đệm dữ liệu. Quy trình này được phần cứng thực hiện tuần tự qua 3 bước:

- Chèn bit 1: Thêm một bit '1' ngay sau bit cuối cùng của thông điệp thô.

- Chèn bit 0: Tiếp tục chèn các bit '0' sao cho tổng độ dài chuỗi tại bước này đồng dư với 448 theo modulo 512. (Tức là chứa lại đúng 64 bit trống ở cuối khối 512-bit cuối cùng).
- Chèn chiều dài thông điệp: Tại 64 bit cuối cùng, chèn giá trị nhị phân biểu diễn độ dài L của thông điệp gốc theo định dạng Big-Endian.

Sau quá trình này, thông điệp sẽ được phân rã thành N khối 512-bit ($M_1, M_2, \dots, M_N$) để lần lượt đưa vào lõi nén.



Hình 2. Sơ đồ FSM điều khiển bộ đệm dữ liệu

Điều khiển của module được hiện thực hóa bằng một mạch trạng thái hữu hạn (FSM) xử lý tuần tự qua 6 trường hợp trạng thái (localparam) để bao quát toàn bộ các kịch bản biên của thuật toán:

Quá trình bắt đầu ở trạng thái STATE_RECEIVE. Tại đây, mỗi byte dữ liệu nhận được từ tín hiệu data_in sẽ được ghi tuần tự vào bộ đệm khối (block buffer), đồng thời biến đếm tổng số byte đã nhận (total_bytes) được tăng lên. FSM tiếp tục duy trì trạng thái này cho đến khi xuất hiện cạnh lên của xung clock mà tại đó cả hai tín hiệu byte_valid và byte_last đều bằng 1, tức là byte hiện tại là byte cuối cùng của toàn bộ thông điệp. Khi nhận được byte cuối cùng, FSM không phát khối dữ liệu ngay mà trước hết đánh giá giá trị của byte_count, tức số byte dữ liệu hiện đang chứa trong block cuối cùng.

Trường hợp thứ nhất, nếu $\text{byte_count} < 55$, nghĩa là trong block hiện tại vẫn còn đủ

không gian để chèn toàn bộ phần đệm SHA-256 gồm bit '1', các bit '0' và trường độ dài 64 bit. Khi đó FSM chuyển sang STATE_EMIT_LAST1, tạo một block 512 bit duy nhất bằng cách giữ nguyên phần dữ liệu thực tế, chèn byte 0x80 ngay sau dữ liệu, điền các bit 0 vào phần còn lại và đặt trường độ dài 64 bit ở cuối block. Sau khi phát block này, tín hiệu message_done được kích hoạt vì toàn bộ thông điệp đã được xử lý hoàn tất.

Trường hợp thứ hai, nếu $55 \leq \text{byte_count} < 63$, block hiện tại không còn đủ chỗ để chứa trường độ dài 64 bit. Khi đó FSM phải tạo hai block. Đầu tiên FSM chuyển sang STATE_EMIT_LASTA, phát block thứ nhất chứa dữ liệu gốc, byte 0x80 và các bit 0 lấp đầy đến hết block. Sang chu kỳ tiếp theo, FSM chuyển sang STATE_EMIT_LASTB, phát block thứ hai chỉ gồm các bit 0 và trường độ dài 64 bit ở cuối block. Sau khi block thứ hai được phát ra, tín hiệu message_done được kích hoạt để báo kết thúc xử lý thông điệp.

Trường hợp thứ ba, nếu $\text{byte_count} == 63$, tức block hiện tại chỉ còn đúng 1 byte trống. Byte trống này được sử dụng để ghi giá trị 0x80, nhưng không còn không gian nào cho trường độ dài. Do đó FSM chuyển sang STATE_EMIT_FULL để phát block đầu tiên đã đầy đủ 512 bit. Đồng thời một cờ pad_after_full được thiết lập nhằm ghi nhớ rằng vẫn còn một block đệm thứ hai cần phát. Ở chu kỳ tiếp theo FSM chuyển sang STATE_PAD, tạo block thứ hai gồm toàn bộ các bit 0 ngoại trừ trường độ dài 64 bit ở cuối block. Sau khi block này được phát, tín hiệu block_done được kích hoạt và message_done được hạ xuống để kết thúc quá trình.

Với mỗi khối 512-bit được nạp vào, thuật toán SHA-256 yêu cầu thực hiện chuỗi 64 vòng biến đổi (từ $t = 0$ đến 63). Tuy nhiên, thay vì xử lý tuần tự 1 vòng/chu kỳ, lõi phần cứng của đề tài được tối ưu bằng kiến trúc xử lý song song 2 vòng lặp trong một chu kỳ xung nhịp.

Tại mỗi chu kỳ, bộ nhớ sẽ truy xuất đồng thời cặp hằng số vòng (K_0, K_1) và từ dữ liệu mở rộng (W_0, W_1). Nguồn dữ liệu kép này được cấp phát cho hai khối logic tổ hợp (Round 0 và Round 1) mắc nối tầng liên tiếp. Nhờ đó, trạng thái của 8 thanh ghi làm việc (từ a đến h) được đi xuyên qua các hàm logic phi tuyến (Sigma, Choose, Majority) của cả 2 vòng và cập nhật kết quả ngay trong một nhịp clock.

Sau khi hoàn tất 64 vòng lặp (lúc này thực tế chỉ tồn 32 chu kỳ lõi), giá trị của 8 thanh ghi làm việc sẽ được cộng tích lũy (modulo 2^{32}) với giá trị băm nền trước đó (lưu trong mảng thanh ghi (H_0 đến H_7)). Đặc tính phụ thuộc dữ liệu cao này đặt ra yêu cầu phải

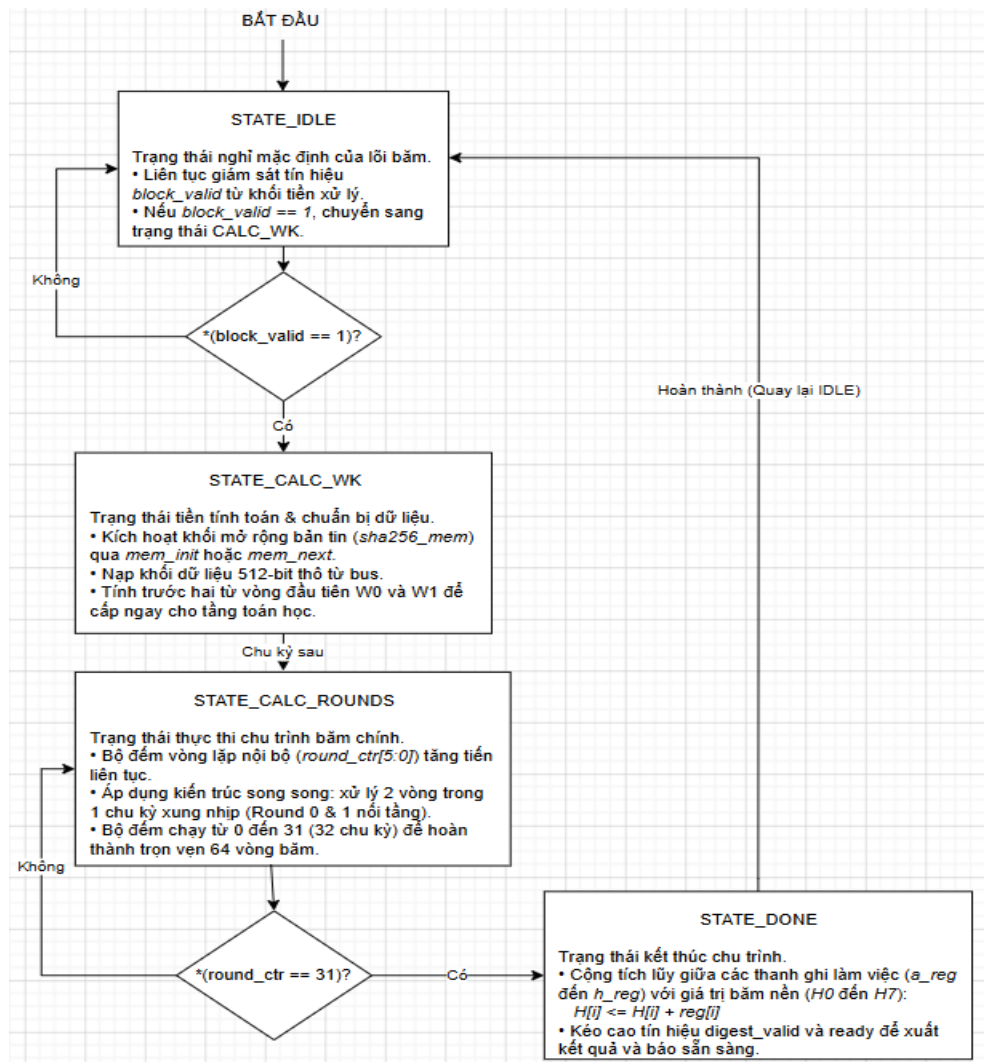
thiết kế một bộ điều khiển FSM chặt chẽ. Nhờ FSM tối ưu kết hợp với lõi tính toán song song, tổng thời gian xử lý trọn vẹn một khối 512-bit được rút ngắn tối đa chỉ còn 36 chu kỳ máy.

- **sha_message_controller:** Khối tiền xử lý, tự động chèn bit đệm (padding) và đóng gói thông điệp thô thành các khối 512-bit chuẩn.



Hình 3. Sơ đồ khối sha_message_controller

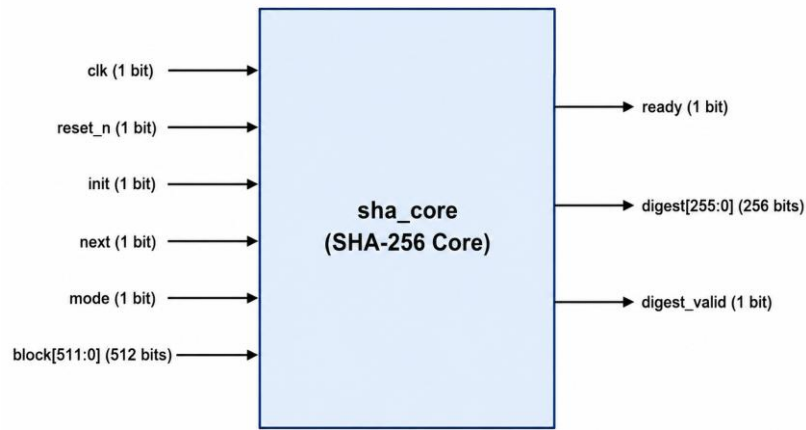
2.1.3. Quy trình xử lý trong khối Core (sha256_core):



Hình 4. Sơ đồ FSM điều khiển lõi băm SHA-256

Mạch điều khiển trung tâm của khối lõi được hiện thực hóa bằng một mạch trạng thái hữu hạn (FSM) gồm 4 trường hợp trạng thái chính để điều phối luồng tính toán số học:

- Trạng thái IDLE: Trạng thái nghỉ mặc định của lõi băm. Tại đây, mạch liên tục giám sát tín hiệu `block_valid` từ khối tiền xử lý. Khi `block_valid` lên mức cao, FSM sẽ chuyển sang trạng thái `CALC_WK`.
- Trạng thái `CALC_WK`: Trạng thái tiền tính toán và chuẩn bị dữ liệu. Mạch kích hoạt khối mở rộng bản tin (`sha256_mem`) thông qua việc phát các tín hiệu khởi tạo `mem_init` hoặc dịch chuyển dữ liệu `mem_next`. Khối nạp dữ liệu 512-bit thô từ bus vào và tính toán trước hai từ vòng đầu tiên `W0` và `W1` để cấp phát ngay cho tầng toán học tổ hợp.
- Trạng thái `CALC_ROUNDS`: Trạng thái thực thi chu trình băm chính. Bộ đếm vòng lặp nội bộ (`round_ctr[5:0]`) tăng tiến liên tục để điều khiển luồng dữ liệu chạy qua hai khối mạch tổ hợp toán học `sha256_round_comb` (Round 0 và Round 1) được mắc nối tầng liên tiếp. Do mạch áp dụng kiến trúc mở rộng xử lý song song 2 vòng trong 1 chu kỳ xung nhịp, bộ đếm `round_ctr` chỉ cần chạy từ giá trị 0 đến 31 (tương đương 32 chu kỳ máy) là hoàn thành xong trọn vẹn 64 vòng băm tiêu chuẩn của thuật toán.
- Trạng thái `DONE`: Trạng thái kết thúc chu trình. Lõi băm thực hiện phép cộng tích lũy giữa giá trị hiện tại của các thanh ghi làm việc (`a_reg` đến `h_reg`) với giá trị băm nền lưu trong các thanh ghi kết quả (`H0` đến `H7`) theo đúng công thức: $H[i] \leftarrow H[i] + reg[i]$. Khi phép cộng kết thúc, mạch kéo cao tín hiệu `digest_valid` và `ready` để xuất kết quả và báo sẵn sàng cho khối tiếp theo.
- **sha_core**: Hạt nhân xử lý chính, được tối ưu hóa kiến trúc song song.



Hình 5. Sơ đồ khối sha256_core

2.1.4. Khối tra cứu hằng số vòng (sha256_constants_k):

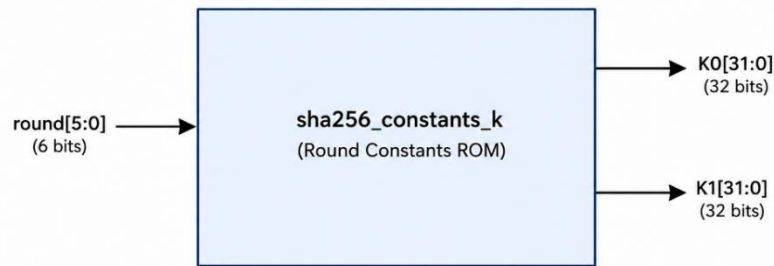
Khối tra cứu hằng số vòng (sha256_constants_k) hoạt động như một bộ nhớ chỉ đọc (ROM) để lưu trữ 64 hằng số toán học 32-bit của thuật toán SHA-256. Các hằng số này được xác định theo tiêu chuẩn FIPS 180-4 của NIST [1].

Do kiến trúc hệ thống lựa chọn phương án tính toán song song 2 vòng (rounds) trong cùng một chu kỳ xung nhịp, khối này có chức năng cung cấp đồng thời hai hằng số độc lập K0 và K1 cho tầng mạch tổ hợp phía sau. Cơ chế này giúp đáp ứng tức thời dữ liệu hằng số mà không tạo ra độ trễ lưu trữ qua thanh ghi trung gian.

Mạch logic bên trong module được thiết kế thuần túy là mạch tổ hợp. Dữ liệu ngõ ra K0 và K1 sẽ thay đổi ngay lập tức khi có sự thay đổi ở bus địa chỉ ngõ vào.

Sơ đồ chân giao tiếp của module bao gồm các tín hiệu sau:

- round[5:0] (Ngõ vào - 6 bits): Nhận chỉ số vòng lặp hiện tại từ bộ đếm vòng của mạch FSM trung tâm.
- K0[31:0] (Ngõ ra - 32 bits): Xuất hằng số vòng ứng với chỉ số round hiện tại.
- K1[31:0] (Ngõ ra - 32 bits): Xuất hằng số vòng ứng với chỉ số vòng kế tiếp (round + 1).
- **sha256_constants_k**: ROM tra cứu, xuất song song 2 hằng số vòng (K₀, K₁).



Hình 6. Sơ đồ khối `sha256_constants_k`

2.1.5. Bộ nhớ lịch trình thông điệp (`sha256_mem`):

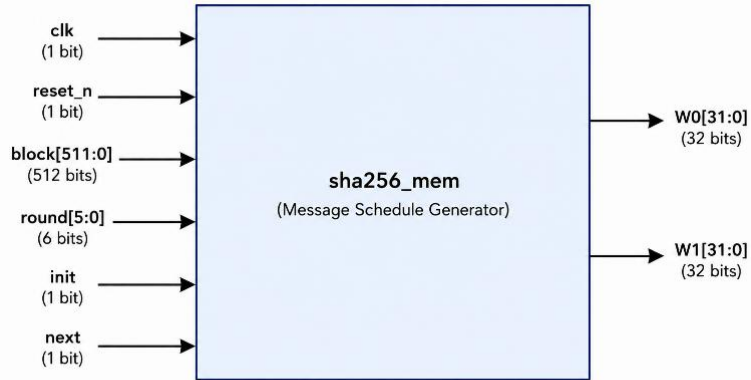
Module `sha256_mem` đóng vai trò là bộ nhớ lịch trình thông điệp bên trong lõi xử lý băm SHA-256. Khối này chịu trách nhiệm lưu trữ tạm thời 16 từ (word) 32-bit của khối dữ liệu 512-bit ngõ vào, đồng thời thực thi các phép toán logic tổ hợp để mở rộng thông điệp, tạo ra các từ vòng $W[t]$ tiếp theo theo đúng tiêu chuẩn của thuật toán SHA-256 [1].

Để đáp ứng và hỗ trợ tối ưu cho kiến trúc tăng tốc phần cứng xử lý song song 2 vòng lặp (2 rounds) trong cùng một chu kỳ xung nhịp, module `sha256_mem` được thiết kế để tính toán và kết xuất đồng thời hai từ vòng 32-bit độc lập là $W0$ và $W1$ trong mỗi chu kỳ clock. Cơ chế này giúp cung cấp tức thời dữ liệu cho hai tầng toán học nối tiếp phía sau mà không gây trễ thất nút cổ chai cho luồng nạp dữ liệu.

Dựa trên kiến trúc phần cứng, các chân tín hiệu giao tiếp nội bộ của module `sha256_mem` được mô tả chi tiết qua sơ đồ giao tiếp và đặc tả kỹ thuật cụ thể:

- `clk` (Ngõ vào - 1 bit): Xung nhịp đồng bộ điều khiển hệ thống thanh ghi bên trong.
- `reset_n` (Ngõ vào - 1 bit): Tín hiệu khởi tạo lại mạch, tích cực ở mức thấp.
- `block[511:0]` (Ngõ vào - 512 bits): Bus dữ liệu song song tiếp nhận khối bản tin 512-bit từ khối tiền xử lý.
- `round[5:0]` (Ngõ vào - 6 bits): Chỉ số đếm vòng lặp hiện tại tại từ bộ điều khiển FSM trung tâm.
- `init` (Ngõ vào - 1 bit): Tín hiệu điều khiển khởi tạo, nạp trực tiếp khối dữ liệu 512-bit vào mảng thanh ghi nội bộ.
- `next` (Ngõ vào - 1 bit): Tín hiệu điều khiển dịch chuyển mảng thanh ghi và cập nhật từ vòng mới khi chỉ số vòng lặp lớn hơn hoặc bằng 16.
- `W0[31:0]` (Ngõ ra - 32 bits): Từ vòng thứ nhất cấp phát cho khối tính toán

- Round 0.
- $W1[31:0]$ (Ngõ ra - 32 bits): Từ vòng thứ hai cấp phát cho khối tính toán Round 1.
 - **sha256_mem**: Lưu khối 512-bit, tính toán và xuất đồng thời 2 từ vòng (W_0 , W_1) mỗi nhịp clock.



Hình 7. Sơ đồ khối sha256_mem

2.1.6. Mạch tổ hợp vòng băm (sha256_round_comb):

Module sha256_round_comb là mạch logic tổ hợp thực thi trọn vẹn một bước tính toán vòng (round transformation) của thuật toán SHA-256. Chức năng cốt lõi của module là nhận giá trị hiện tại của 8 thanh ghi làm việc (từ a đến h) cùng giá trị tổng giữa hằng số vòng và từ vòng ($W+K$), sau đó thực hiện xáo trộn qua các hàm logic phi tuyến để tạo ra trạng thái kế tiếp.

Trong kiến trúc thiết kế này, module được xây dựng thuần túy là mạch tổ hợp (combinational logic), không chứa các phần tử lưu trữ thanh ghi. Điều này cho phép hai khối sha256_round_comb có thể mắc nối tầng liên tiếp bên trong khối sha_core, giúp hệ thống tính toán được 2 vòng băm trong chỉ một chu kỳ xung nhịp, tăng gấp đôi hiệu suất xử lý so với kiến trúc tuần tự.

Mạch logic bên trong thực hiện tính toán song song các hàm băm tiêu chuẩn thông qua các bước logic sau:

Tính toán các hàm phi tuyến và xoay bit:

- Hàm S1: Thực hiện xoay phải vòng bit của biến e_{in} tại các khoảng 6, 11 và 25 bit, sau đó thực hiện phép toán XOR giữa các kết quả.

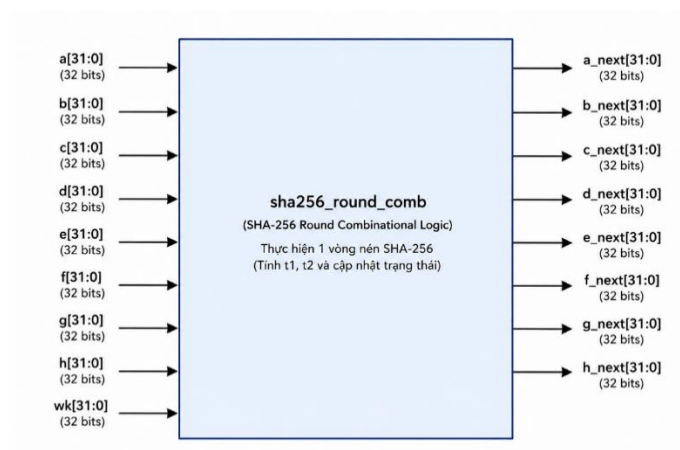
- Hàm Ch (Choose): Thực hiện lựa chọn bit giữa f_in và g_in dựa trên trạng thái của bit e_in .
- Hàm S0: Thực hiện xoay phải vòng bit của biến a_in tại các khoảng 2, 13 và 22 bit, sau đó XOR các kết quả lại với nhau.
- Hàm Maj (Majority): Thực hiện logic tìm bit chiếm đa số giữa ba biến đầu vào a_in , b_in và c_in .

Tính toán các giá trị tổng trung gian (T1 và T2):

- Biến tạm T1 được tính bằng tổng Modulo 2 mũ 32 của: h_in , S1, Ch, và wk_in .
- Biến tạm T2 được tính bằng tổng Modulo 2 mũ 32 của: S0 và Maj.

Cập nhật trạng thái ngõ ra (Update Logic): Các giá trị ngõ ra được gán trực tiếp thông qua các đường dây tổ hợp theo quy tắc dịch chuyển và cập nhật của SHA-256:

- $h_out = g_in$
 - $g_out = f_in$
 - $f_out = e_in$
 - $e_out = d_in + T1$
 - $d_out = c_in$
 - $c_out = b_in$
 - $b_out = a_in$
 - $a_out = T1 + T2$
- **sha256_round_comb:** Mạch logic tổ hợp xử lý hàm băm. Hai khối này được mắc nối tiếp để đạt tốc độ băm 2 vòng/chu kỳ.



Hình 8. Sơ đồ khối sha256_round_comb

2.1.7. Thiết kế module top (sha256_top):

Chức năng của module là định hình cấu trúc chân tín hiệu chuẩn, thực hiện đồng bộ tín hiệu hệ thống và quản lý luồng dữ liệu vào/ra toàn cục.

Module thực hiện đóng gói toàn bộ các phân khối thiết kế trong chương này thông qua kỹ thuật kết nối cổng (port mapping) trong Verilog:

- Khởi tạo lỗi băm: Gọi module sha_core làm thành phần chính.
- Quản lý đồng bộ: Kết nối các tín hiệu hệ thống (clk, reset_n) đến tất cả các module con.
- Định tuyến dữ liệu: Thiết lập luồng dữ liệu thông qua các bus trung gian để đảm bảo dữ liệu từ ngõ vào data_in được chuyển đến đúng module xử lý tiền bối trước khi đưa vào lõi băm.
- **sha256_top**: Lớp bao bọc tổng, đồng bộ tín hiệu và quản lý luồng dữ liệu toàn cục.



Hình 9. Sơ đồ khối sha256_top

2.1.8. Thiết kế module Avalon (sha_avalon_top):

Đây là module tầng cao nhất (top-level) của hệ thống phần cứng, đóng vai trò là cầu nối (bridge/wrapper) giữa lõi băm phần cứng (sha256_core) và vi xử lý trung tâm (như Nios II) thông qua chuẩn giao tiếp Avalon Memory-Mapped (Avalon-MM).

Chức năng cốt lõi của module là thực hiện Ánh xạ bộ nhớ (Memory-Mapping). Nó dịch các lệnh Đọc (Read) và Ghi (Write) từ bus Avalon thành các tín hiệu điều khiển trực tiếp cho lõi SHA-256. Nhờ khối này, CPU không cần biết về cấu trúc phức tạp bên trong lõi băm, mà chỉ cần tương tác bằng cách ghi/đọc dữ liệu vào các địa chỉ (address) đã được quy định sẵn (thanh ghi dữ liệu, thanh ghi điều khiển, thanh ghi trạng thái).

Module thực hiện đóng gói toàn bộ hệ thống bằng kỹ thuật thiết kế Register File (Tập

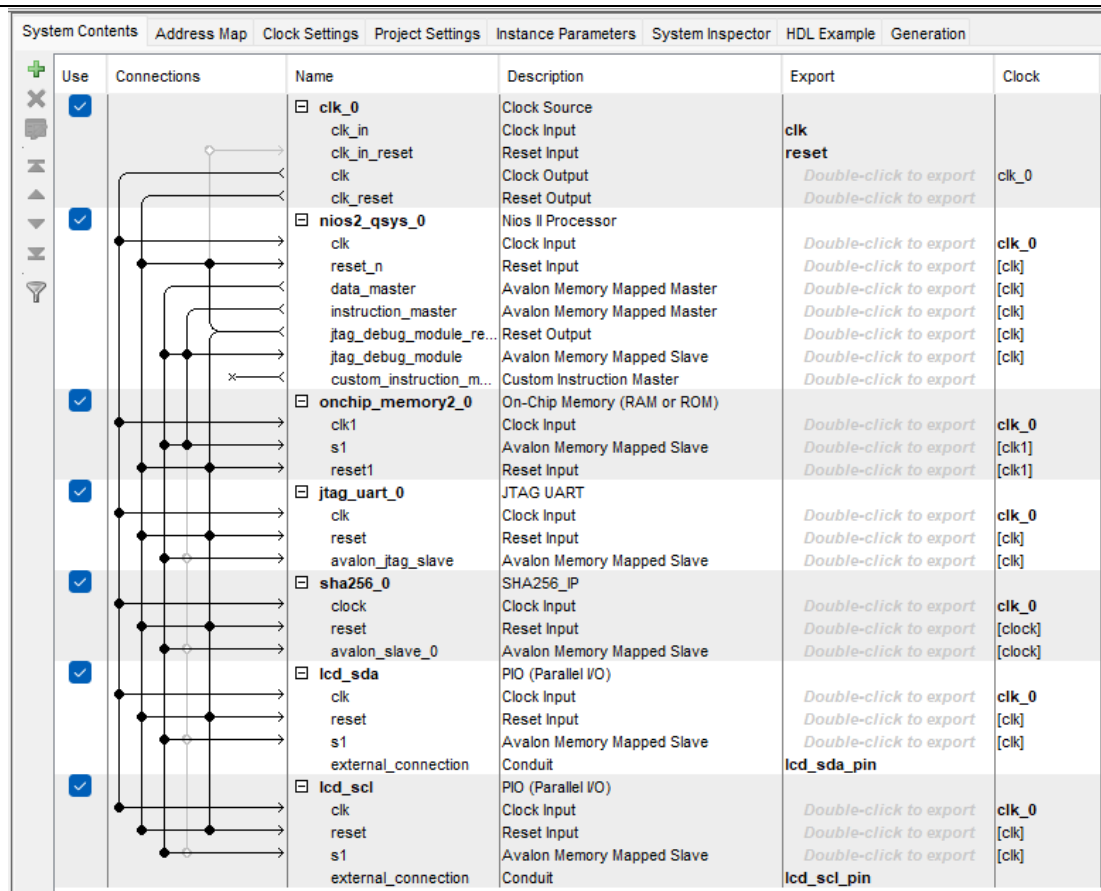
thanh ghi) kết hợp khởi tạo thành phần con (port mapping):

- Bộ giải mã địa chỉ: Là trái tim của khối Avalon, định tuyến luồng dữ liệu dựa trên giá trị của chân address:
 - + Từ địa chỉ 0 đến 15: Ánh xạ vào 16 thanh ghi block_in (mỗi thanh ghi 32-bit). Dữ liệu từ writedata được chót vào đây làm nguyên liệu cho lõi băm.
 - + Địa chỉ 16 (Ctrl/Status):
 - Ghi (Write): Kích hoạt tín hiệu init để bắt đầu băm.
 - Đọc (Read): Đẩy các cờ trạng thái (digest_valid, ready, done_reg) lên bus readdata để CPU thăm dò.
 - + Từ địa chỉ 17 đến 24: Ánh xạ vào 8 thanh ghi digest_out. CPU đọc các địa chỉ này để thu thập mã băm 256-bit cuối cùng.
- Khởi tạo lõi băm: Gọi module sha256_core làm lõi xử lý trung tâm. Toàn bộ các thanh ghi Block, Digest và các dây tín hiệu điều khiển từ bộ giải mã được nối trực tiếp vào các port của lõi này.
- Quản lý đồng bộ: Tín hiệu clk và reset_n từ bus Avalon được phân phối xuyên suốt, đảm bảo bộ giải mã thanh ghi và lõi sha256_core hoạt động đồng nhịp một cách nghiêm ngặt.
- **sha256_avalon_top**: Cầu nối giao tiếp Avalon-MM, giải mã địa chỉ để vi xử lý Nios II điều khiển và đọc/ghi dữ liệu với lõi băm.



Hình 10. Sơ đồ khối sha256_avalon_top

2.2. Thiết kế hệ thống Qsys



Hình 11. Sơ đồ hệ thống Qsys

- NIOS II: Điều khiển phần mềm
 - + Tiền xử lý dữ liệu: cắt chuỗi, đệm bit.
 - + Điều khiển luồng: ra lệnh bắt đầu (init), sử dụng cơ chế polling.
 - + Đọc kết quả từ lõi SHA 256 và xuất ra màn hình LCD.
- Onchip Memory: Bộ nhớ chương trình, trung gian lưu trữ ví dụ dữ liệu thô từ code C block[64] và dữ liệu được băm từ code Verilog digest[32].
- Jtag: Dùng để hiển thị kết quả lên màn hình console.
- Bus 32 bit:
 - + SHA256 có word size xử lý là 32 bit phù hợp với Bus 32 bit.
 - + NIOS II là kiến trúc 32 bit.
- 2 PIO: SDA, SCL dùng để giao tiếp I2C với LCD.

2.3. Thiết kế phần mềm

- Thanh ghi sử dụng: Tổng cộng 25 thanh ghi
 - + 0 – 15: nạp dữ liệu 512 bit tức là 16 word vào phần cứng
 - + 16: phép gán sẽ là chế độ ghi, bật cờ init cho phép hàm băm thực hiện, phép so sánh sẽ là chế độ đọc, bật cờ done báo cho
 - + 17 – 24: đọc kết quả 256 bit tức là 8 word từ hàm băm sang phần mềm.
- Cơ chế xử lý:
 - + Chuyển đổi Endianness: Phần mềm sử dụng hàm `be_word()` để sắp xếp lại byte dữ liệu, chuyển đổi từ kiến trúc Little-Endian của vi xử lý NIOS II sang Big-Endian bắt buộc của thuật toán SHA-256.
 - + Padding (Đệm dữ liệu): Thực hiện đóng gói thông điệp thô thành khối 512-bit chuẩn FIPS 180-4 bằng cách chèn byte 0x80, điền các byte 0x00 và gán giá trị độ dài thông điệp gốc (64-bit) vào cuối khối.
- Luồng xử lý:
 - + Tiền xử lý: Nhận thông điệp, thực hiện thuật toán Padding.
 - + Truyền tải: Ghi dữ liệu 512-bit đã đệm vào thanh ghi [0-15].
 - + Kích hoạt: Ghi lệnh INIT vào thanh ghi [16] để phần cứng bắt đầu tính toán.
 - + Đồng bộ (Polling): Vi xử lý liên tục đọc thanh ghi [16] để chờ cờ DONE bật sáng.
 - + Trích xuất & Hiển thị: Đọc 256-bit kết quả từ thanh ghi [17-24], chuyển đổi sang định dạng chuỗi Hex 64 ký tự và xuất ra giao diện UART hoặc LCD.

Chương 3: MÔ PHÒNG VÀ TỔNG HỢP

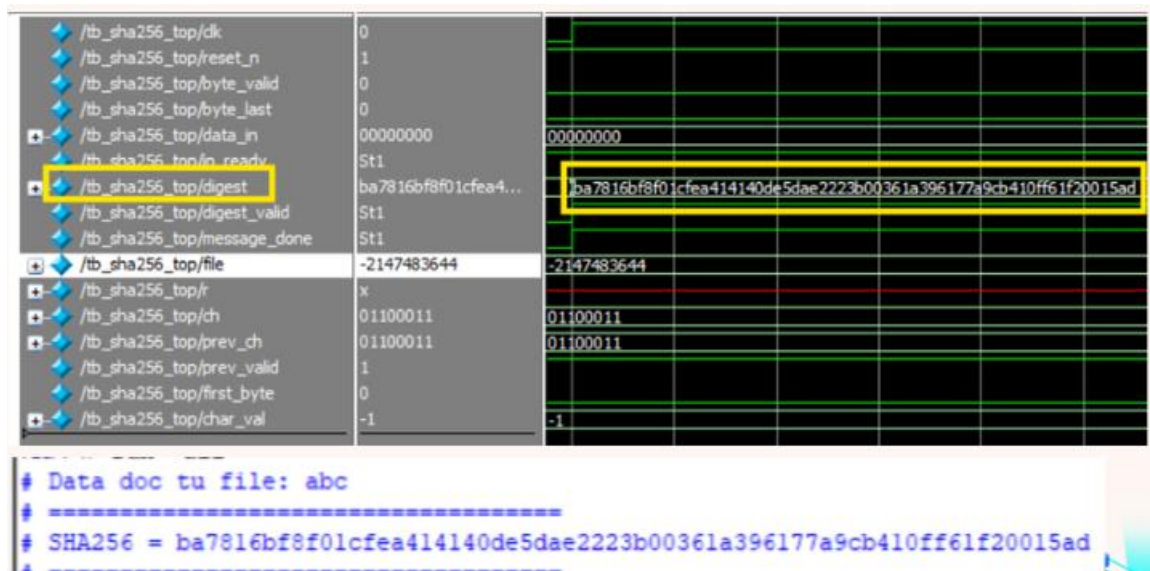
3.1. Kết quả mô phỏng

3.1.1. Thiết lập kịch bản kiểm chứng

Để kiểm chứng chức năng của lõi SHA-256, một Testbench toàn diện (tb_sha256_top) đã được thiết lập với các đặc tính kỹ thuật sau:

- Phương pháp nạp dữ liệu: Thay vì gán cứng dữ liệu, thiết kế sử dụng phương pháp đọc file (input.txt) thông qua lệnh \$fgetc, cho phép kiểm tra linh hoạt nhiều định dạng dữ liệu khác nhau.
- Cơ chế bắt tay: Sử dụng một task chuyên dụng là send_byte để đảm bảo dữ liệu chỉ được gửi khi tín hiệu in_ready ở mức cao. Điều này mô phỏng sát thực tế hoạt động của các bus giao tiếp dữ liệu tốc độ cao.
- Xử lý tín hiệu: Testbench thực hiện logic kiểm soát luồng byte, bao gồm xử lý ký tự cuối (byte_last) và loại bỏ các ký tự điều khiển (CR, LF) không cần thiết để đảm bảo dữ liệu đưa vào thuật toán băm hoàn toàn chính xác theo chuẩn NIST.

3.1.2. Phân tích dạng sóng và kết quả đầu ra



Hình 12. Sóng waveform

Kết quả mô phỏng được đánh giá dựa trên các tiêu chí:

1. **Luồng dữ liệu ổn định:** Tín hiệu byte_valid và data_in chuyển đổi nhịp nhàng, dữ liệu được hệ thống tiếp nhận đúng theo từng xung nhịp clk sau khi chờ in_ready được xác nhận.

2. **Xử lý tín hiệu kết thúc:** Cờ byte_last được kích hoạt chính xác tại byte cuối cùng của bản tin, cho phép FSM điều khiển của module sha_core chuyển sang giai đoạn tổng kết hash.
3. **Độ chính xác của kết quả:** Sau khi tín hiệu digest_valid được bật, giá trị digest hiển thị tại Tcl Console được đối chiếu với các công cụ tính hash chuẩn (ví dụ: sha256sum trên Linux hoặc các trang web tính hash trực tuyến), cho thấy sự trùng khớp hoàn toàn. Với dữ liệu đầu vào từ file, hệ thống trả về kết quả định dạng Hex.

3.2. Kết quả tổng hợp

3.2.1. Phân tích tài nguyên hệ thống

Thiết kế lõi băm SHA-256 được tổng hợp trên chip FPGA DE2 kết quả sử dụng tài nguyên logic được ghi nhận cụ thể như sau:

Resource	Usage
Logic Elements (LEs)	5,056 / 33,216 (15%)
LUTs	4,388
Flip-Flops (Registers)	2,554 / 34,593 (7%)
I/O Pins	38 / 475 (8%)
BRAMs (M4K Blocks)	70 / 105 (67%)
DSP Blocks (Embedded Multipliers)	0 / 70 (0%)

Bảng 3: Bảng khai báo tài nguyên sử dụng

Nhận xét: Kết quả tổng hợp cho thấy thiết kế sử dụng 4.388 LUTs và 2.554 thanh ghi, tương ứng khoảng 15% tài nguyên logic của FPGA, cho thấy mức độ khai thác phần cứng hợp lý và còn nhiều dư địa mở rộng. Khối bộ nhớ BRAM sử dụng 70/105 khối (67%), phản ánh việc lưu trữ các hằng số và dữ liệu trung gian của thuật toán SHA-256 được thực hiện hiệu quả. Số lượng chân I/O sử dụng chỉ chiếm 8% tổng số chân khả dụng, không gây áp lực lên tài nguyên ngoại vi của hệ thống. Đồng thời, thiết kế không sử dụng các khối DSP chuyên dụng, cho thấy thuật toán được hiện thực hoàn toàn bằng tài nguyên logic và bộ nhớ trên FPGA. Tổng thể, mức sử dụng tài nguyên đáp ứng tốt yêu cầu triển khai SoC SHA-256 với khả năng mở rộng và tối ưu phần cứng trong tương lai.

3.2.2. Đánh giá thời gian thực

Dựa trên báo cáo phân tích thời gian thực tế, thiết kế đáp ứng đầy đủ các ràng buộc về thời gian:

- Worst Negative Slack (WNS): 1.468 ns (dương), khẳng định thiết kế hoàn toàn không vi phạm ràng buộc thời gian (Setup time).

Slow Model Setup Summary			
	Clock	Slack	End Point TNS
1	CLOCK_50	1.468	0.000

Hình 13. Worst Negative Slack

- Tần số vận hành tối đa (Fmax): Đạt ngưỡng khoảng 53.96 MHz.

Slow Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	53.96 MHz	53.96 MHz	CLOCK_50	

Hình 14. Chỉ số Fmax

- Worst Hold Slack (WHS): 0.215 ns (dương), đảm bảo tính ổn định của dữ liệu tại các cạnh xung nhịp.

Fast Model Hold Summary			
	Clock	Slack	End Point TNS
1	CLOCK_50	0.215	0.000

Hình 15. Worst Hold Slack

Kết luận: Thiết kế đạt trạng thái "Timing Met", toàn bộ đường dẫn tín hiệu (endpoints) đều nằm trong ngưỡng an toàn, khẳng định độ tin cậy khi vận hành thực tế. Tuy nhiên, cần lưu ý rằng tần số vận hành tối đa hiện tại là hệ quả của chiến lược ưu tiên **Thông lượng (Throughput)**.

3.2.3. Hiệu năng tính toán

Thiết kế sử dụng kiến trúc xử lý song song, kết hợp hai bộ tính toán vòng băm (sha256_round_comb) trong cùng một chu kỳ xung nhịp. Để duy trì luồng dữ liệu liên tục cho hai khối xử lý này, hệ thống đã được tối ưu hóa cơ chế truy xuất bộ nhớ: tại mỗi chu kỳ, hệ thống thực hiện đồng thời hai thao tác đọc từ bộ nhớ từ vòng (W) và hai thao tác đọc hằng số tương ứng (K).

Cơ chế này giúp rút ngắn quy trình băm một khối dữ liệu 512-bit xuống chỉ còn 36 chu kỳ xung nhịp, tối ưu hóa đáng kể so với kiến trúc tuần tự 64 chu kỳ tiêu chuẩn. Sự kết hợp giữa khả năng truy xuất bộ nhớ song song và logic tổ hợp 2 vòng đã tạo nên một kiến

trúc tăng tốc hiệu quả, cân bằng hài hòa giữa tài nguyên phần cứng và tốc độ xử lý thực tế.

3.2.4. Thông lượng

Thông lượng của thiết kế được tính dựa trên kích thước khối dữ liệu đầu vào, tần số hoạt động tối đa và số chu kỳ xử lý cần thiết cho một khối 512 bit. Công thức tính thông lượng được biểu diễn như sau:

$$\text{Throughput} = (\text{Data_Size} \times \text{Fmax}) / N$$

Trong đó:

$$\text{Data_Size} = 512 \text{ bit}$$

$$\text{Fmax} = 53.96 \text{ MHz}$$

$$N = 36 \text{ chu kỳ (cho một block)}$$

Thay các giá trị vào công thức, ta có:

$$\text{Throughput} = (512 \times 53.96) / 36$$

$$\text{Throughput} \approx 767.43 \text{ Mbps}$$

Như vậy, thiết kế đạt thông lượng xấp xỉ 767.43 Mbps. Kết quả này cho thấy kiến trúc xử lý hai vòng SHA-256 trong mỗi chu kỳ clock giúp giảm số chu kỳ xử lý cho một khối dữ liệu 512 bit so với kiến trúc xử lý tuần tự một vòng mỗi chu kỳ. Thiết kế nhờ đó cải thiện hiệu năng xử lý nhưng vẫn duy trì mức sử dụng tài nguyên phần cứng hợp lý, phù hợp với mục tiêu cân bằng giữa tốc độ và diện tích phần cứng của đồ án.

3.3. Mô phỏng và kiểm chứng thực thi trên vi xử lý Nios II

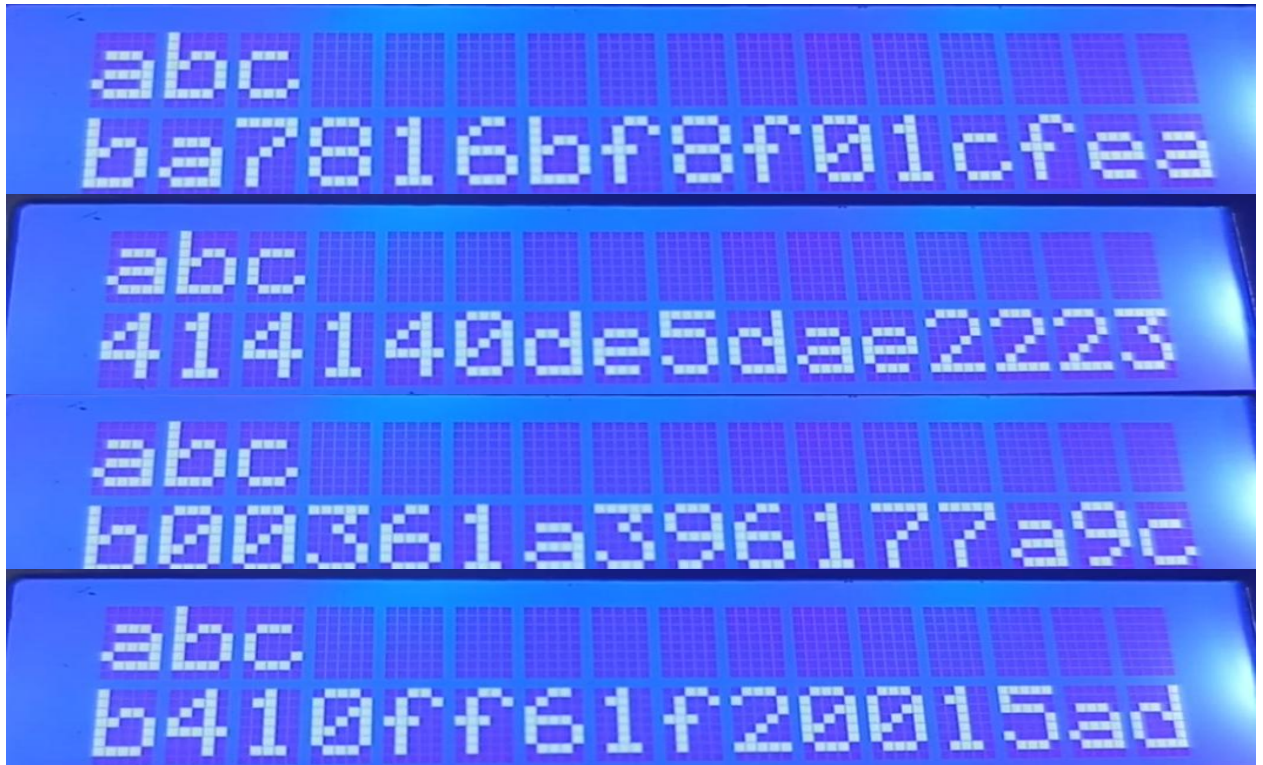
Quá trình đồng bộ phần cứng - phần mềm (HW/SW Co-verification) trên Nios II đã khẳng định hệ thống SoC hoạt động chính xác và cực kỳ ổn định, thể hiện qua 3 yếu tố cốt lõi:

- **Xử lý dữ liệu an toàn:** Hệ thống vượt qua tập kiểm thử 100 chuỗi ký tự đa dạng. Lóp phần mềm tự động lọc và bỏ qua các chuỗi vượt ngưỡng 55 ký tự (chiến lược Khối đơn), ngăn chặn triệt để lỗi tràn bộ đệm hay treo hệ thống.
- **Giao tiếp HW/SW tối ưu:** Nios II nạp nguyên liệu và điều khiển lỗi băm tron tru. Nhờ phần cứng băm siêu tốc (hoàn thành trong 36 chu kỳ), vòng lặp chờ (polling) của CPU thoát gần như tức thời, không tạo ra nút thắt cổ chai cho phần mềm.
- **Xuất liệu chính xác & Trực quan:** Mã băm 256-bit trả về trùng khớp 100% với các công cụ chuẩn khi đối chiếu trên Terminal. Đồng thời, driver I2C Bit-

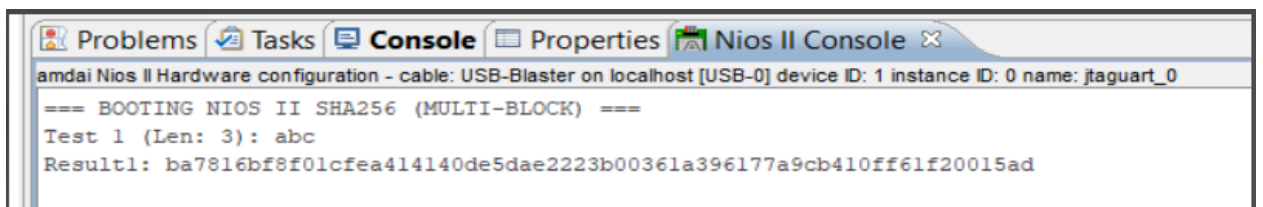
Bangleng hoạt động hoàn hảo, giúp màn hình LCD 16x2 lật trang tự động và hiển thị chuỗi kết quả rõ ràng, độc lập với máy tính.

3.4. Mô phỏng trên kit :

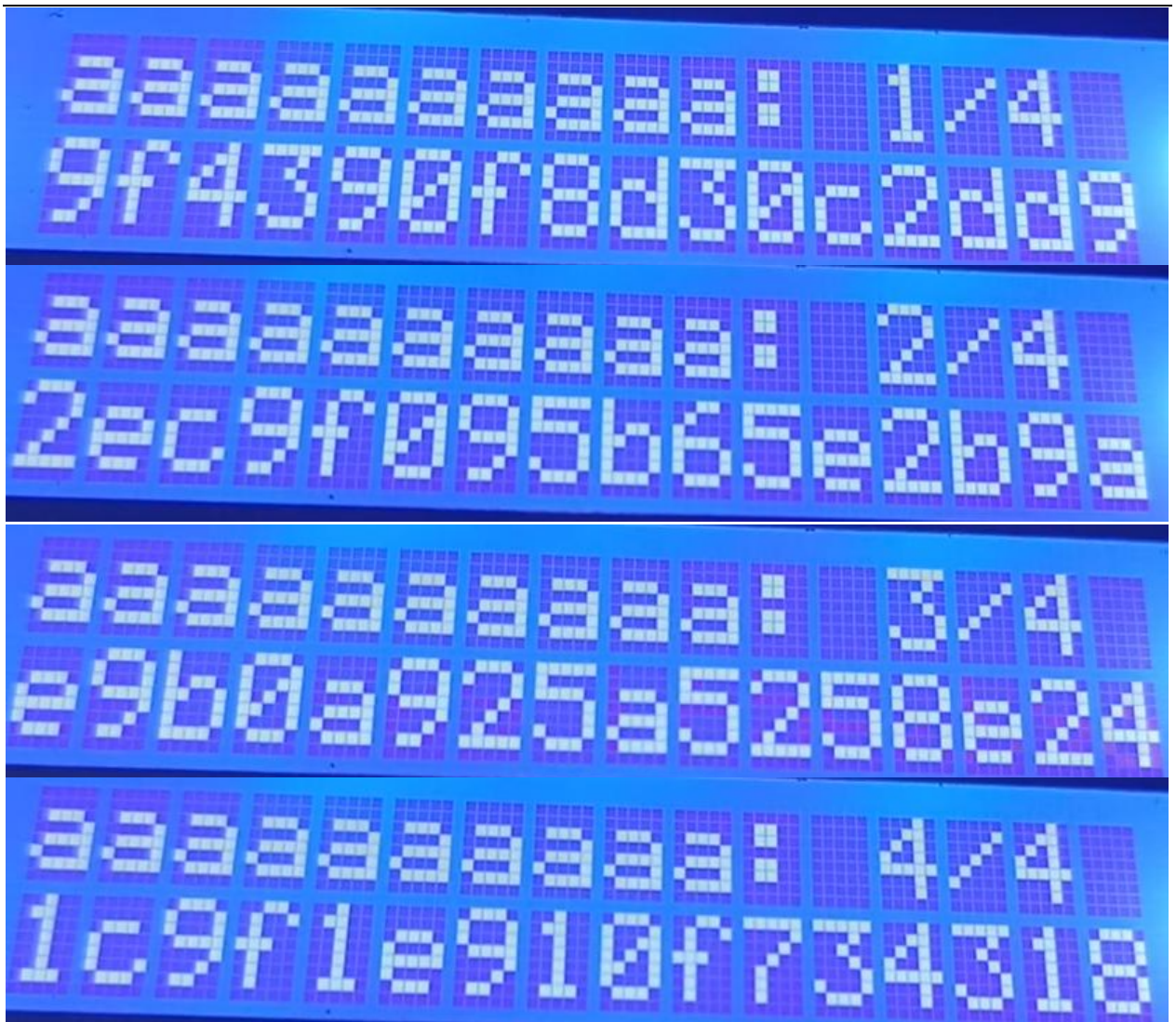
<https://drive.google.com/drive/folders/14uE3dAYy0sBqEsH-UJnqtbJIU5rKGgCd?usp=sharing>



Hình 16. Demo 3 bytes ký tự abc (Single-Block)



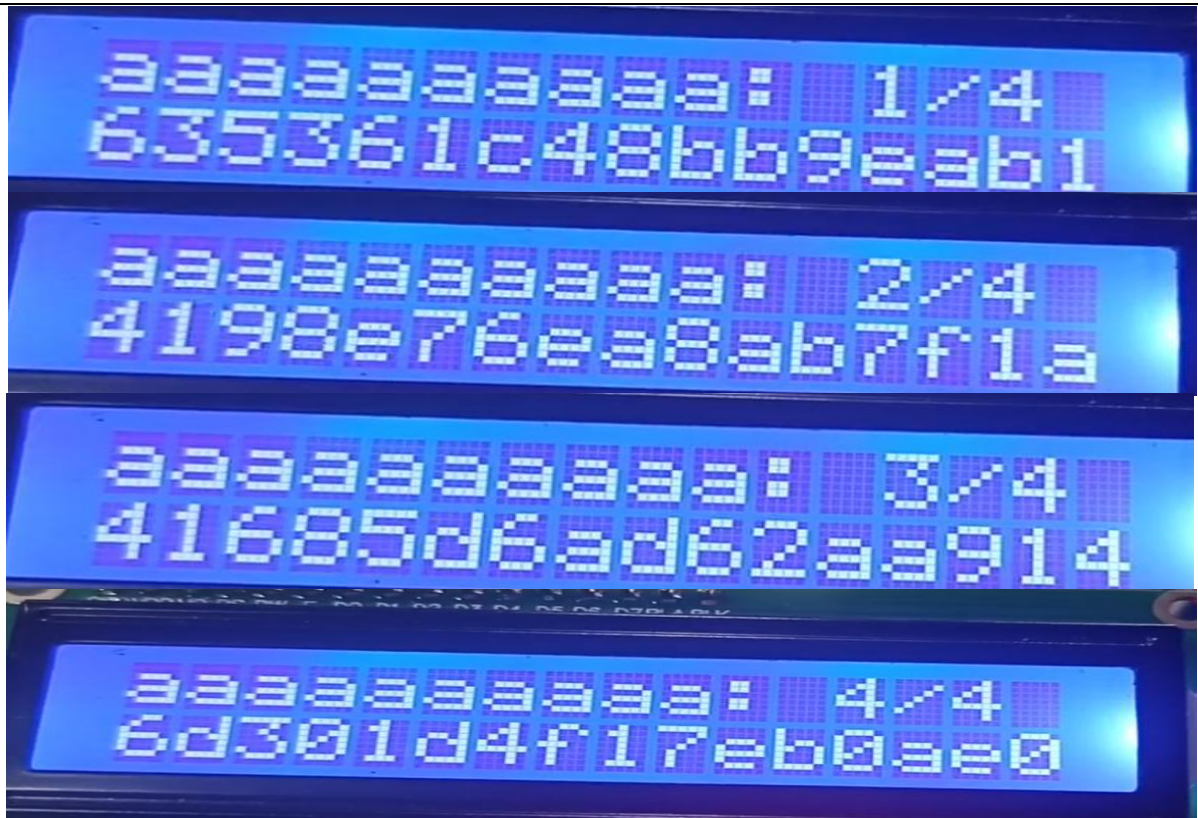
Hình 17. Demo 3 bytes ký tự abc hiện trên console (Single-Block)



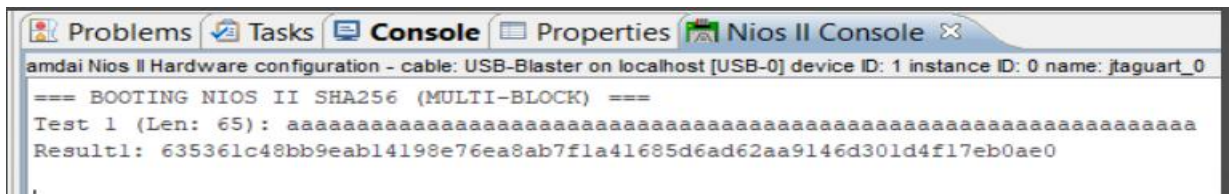
Hình 18. Demo 55 bytes toàn ký tự a (Boundary Single-Block)



Hình 19. Demo 55 bytes toàn ký tự a hiện trên console (Boundary Single-Block)



Hình 20. Demo 65 bytes toàn ký tự a (Multi-Block)



Hình 21. Demo 65 bytes toàn ký tự a hiện trên console (Multi-Block)

Chương 4: KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. Kết quả đạt được

Sau quá trình nghiên cứu và thực hiện, đồ án đã hoàn thành các mục tiêu đề ra:

- Về mặt lý thuyết: Hiểu rõ cấu trúc thuật toán SHA-256 và các tiêu chuẩn bảo mật NIST.
- Về mặt kỹ thuật: Xây dựng thành công lõi IP-Core SHA-256 trên nền tảng FPGA. Thiết kế đạt trạng thái "Timing Met" với tần số vận hành Fmax xấp xỉ 54 MHz.
- Về mặt hiệu năng: Tối ưu hóa thành công kiến trúc xử lý song song 2 vòng/chu kỳ, rút ngắn thời gian băm xuống còn 36 chu kỳ, đạt thông lượng 767.43 Mbps. Đây là kết quả khẳng định khả năng làm chủ kỹ thuật thiết kế phần cứng tốc độ cao.

4.2. Hạn chế - Khó khăn

- Về tài nguyên: Do tập trung vào kiến trúc song song để ưu tiên thông lượng, thiết kế tiêu tốn nhiều tài nguyên logic (LUTs, FFs) hơn so với các kiến trúc tuần tự cơ bản.
- Về giới hạn tần số: Việc ghép nối các khối logic tổ hợp phức tạp để xử lý 2 vòng/chu kỳ đã làm gia tăng độ trễ đường tới hạn (Critical Path), từ đó hạn chế khả năng đẩy lên các mức tần số xung nhịp cao hơn nữa.
- Khó khăn trong kiểm chứng: Việc đồng bộ dữ liệu giữa hai khối xử lý song song và quản lý bộ nhớ đệm (mem) đòi hỏi thời gian debug lớn để đảm bảo tính toàn vẹn dữ liệu cho toàn bộ quá trình băm.

Chương 5: HƯỚNG PHÁT TRIỂN

Để nâng cao hơn nữa hiệu năng và khả năng ứng dụng của hệ thống, nhóm đề xuất các hướng phát triển sau:

- Tối ưu hóa Pipeline (Pipelining): Chia nhỏ đường tới hạn (Critical Path) thành các tầng pipeline sâu hơn nhằm tăng tần số vận hành (Fmax) lên được 54 MHz. Nhóm hiểu rằng việc tăng độ sâu pipeline sẽ làm tăng tổng số chu kỳ xử lý (Latency) cho mỗi khối dữ liệu, tuy nhiên điều này sẽ giúp tối ưu hóa thông lượng (Throughput) tổng thể khi hệ thống xử lý các luồng dữ liệu lớn liên tục.
- Tối ưu năng lượng: Nghiên cứu kỹ thuật quản lý năng lượng (Power Management) để giảm thiểu mức tiêu thụ công suất, phù hợp với các ứng dụng di động hoặc thiết bị nhúng.
- Mở rộng thuật toán: Phát triển kiến trúc để hỗ trợ các biến thể khác của SHA-2 (như SHA-512) hoặc các chuẩn băm mới hơn trong tương lai.

Chương 6: KẾT LUẬN

Việc thiết kế và hiện thực hóa lõi băm SHA-256 trên nền tảng FPGA là một quá trình nghiên cứu kỹ thuật chuyên sâu, đòi hỏi sự phối hợp chặt chẽ giữa tư duy thuật toán và kỹ năng tối ưu hóa phần cứng. Qua quá trình khảo sát, phân tích tiêu chuẩn NIST và hiện thực hóa thiết kế, nhóm nghiên cứu đã xây dựng thành công một lõi IP-Core hoạt động ổn định, đáp ứng tốt các yêu cầu về tốc độ và tính toàn vẹn dữ liệu. Ngoài ra nhóm cũng đã tích hợp thành công vào hệ sống SoC

Hệ thống được phát triển với kiến trúc xử lý song song 2 vòng trong một chu kỳ xung nhịp, giúp khắc phục những hạn chế về độ trễ của các kiến trúc tuần tự truyền thống. Kết quả thu được không chỉ khẳng định tính đúng đắn của thiết kế mà còn đạt được mức thông lượng 767.43 Mbps, minh chứng cho sự tối ưu trong việc cân bằng giữa tài nguyên logic và hiệu năng thực thi. Bên cạnh đó, việc thực hiện đồ án đã giúp nhóm củng cố kiến thức về thiết kế hệ thống số, kỹ thuật mô phỏng và phân tích thời gian thực trên môi trường FPGA.

Tuy nhiên, thiết kế vẫn còn một số hạn chế nhất định về mức tiêu thụ tài nguyên và giới hạn tần số vận hành do đặc thù của các đường dẫn tín hiệu phức tạp.

Cuối cùng, nhóm nghiên cứu xin gửi lời cảm ơn chân thành tới thầy Trương Văn Cương đã tận tình hướng dẫn, đưa ra những định hướng quý báu giúp nhóm hoàn thiện đồ án này. Đồng thời, nhóm cũng xin cảm ơn các thành viên đã cùng nhau nỗ lực, phối hợp chặt chẽ để hoàn thành dự án một cách tốt nhất. Nhóm nghiên cứu rất mong nhận được những ý kiến đóng góp từ Quý Thầy Cô và các bạn để sản phẩm ngày càng hoàn thiện hơn.

Hy vọng kết quả nghiên cứu này sẽ đóng góp một phần nhỏ bé vào việc ứng dụng công nghệ phần cứng tăng tốc trong lĩnh vực bảo mật, đồng thời mở ra những hướng tiếp cận mới cho các hệ thống giám sát và mã hóa dữ liệu tốc độ cao trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] J. Strombergson, "sha256-2," Github, 2016. [Online]. Available: <https://github.com/vrosnet/sha256-2/blob/master/README.md>. [Accessed 25 04 2025].
- [2] L. Wagner, "What is SHA-256?," BOOT.DEV, 2025. [Online]. Available: <https://www.boot.dev/blog/computer-science/how-sha-2-works-step-by-step-sha-256/>. [Accessed 16 04 2026].
- [3] V. S. M. Hafizullah Kakar, "Energy-efficient cryptographic optimization for narrowband IoT," *Internet of Things*, 2026.
- [4] L. Hoang, "Hàm băm SHA-256 là gì? Hiểu rõ về thuật toán của SHA-256," fiahub, 2024. [Online]. Available: <https://www.fiahub.com/blog/ham-bam-sha-256/>. [Accessed 20 04 15].
- [5] Zunokie, "Cách hoạt động của SHA-256," VIBLO, 2021. [Online]. Available: <https://viblo.asia/p/cach-hoat-dong-cua-sha-256-1VgZvJPmZAw>. [Accessed 15 04 2026].